TESTNG

Class 10: @Factory Annotation

Complete Beginner-to-Project Notes

Java + TestNG + Selenium | Code walkthroughs | Realistic scenarios | Best practices

Built from your uploaded TestNG Class 10 notes

Includes the Sample1 / FactoryDemoTest examples, constructor-based data, a Selenium product scenario, XML configuration, troubleshooting, exercises, and interview revision.

Learning plan and contents

This guide explains the code and concepts in the uploaded Class 10 notes. The first sections stay close to the lesson examples; later sections add practical guidance so the technique can be used safely in a real automation framework.

What you will learn

- What @Factory means and when TestNG uses it.
- How to return multiple test-class objects from a factory method.
- How the Sample1 and Sample2 examples run when several instances are created.
- How constructor arguments let each instance carry different test data.
- How to structure a Selenium test so each factory-created instance owns its browser session.
- How to register the factory in testng.xml and diagnose common problems.

Contents

Section	Topics
1. Core idea	Factory definition, simple analogy, syntax
2. Source example	Sample1, TestFactory, and invocation counting
3. Multiple classes	FactoryDemoTest with Sample1 x 3 and Sample2 x 1
4. Constructor data	Passing a different parameter set to each instance
5. Selenium example	Factory-created product scenarios with isolated WebDriver
6. Execution and XML	Registering a factory and optional instance-level parallelism
7. Comparisons	@Factory vs @DataProvider vs invocationCount
8. Quality checklist	Best practices, troubleshooting, exercises, interview questions

Core idea: @Factory creates test-class instances. Every returned object can contribute its own @Test methods to the TestNG run. The factory does not automatically make execution parallel.

1. What is the @Factory annotation?

In TestNG, @Factory marks a method that creates and returns test class objects dynamically. TestNG takes the returned objects and discovers their annotated test methods. If the factory returns several instances of the same class, the test methods on those instances can run for each instance.

Simple analogy

Imagine a factory making three copies of the same test worker. Each worker knows how to run the same set of test methods. The workers can be configured with different constructor values, such as a user role, product, region, or account type.

Basic structure

```
public class SampleTest {
    @Test
    public void login() {
        System.out.println("Login test");
    }
}

public class TestFactory {
    @Factory
    public Object[] createTests() {
        return new Object[] {
            new SampleTest(),
            new SampleTest(),
            new SampleTest()
        };
    }
}
```

Understand the syntax

Code	Meaning
@Factory	Tells TestNG that this method supplies test instances.
Object[]	An array that can hold object references; here it holds test-class instances.
new SampleTest()	Creates a new Java object, separate from the other objects.
return	Gives the objects back to TestNG so it can run their @Test methods.

Important: The returned array contains test objects, not test-data strings. Constructor-based test data is covered later.

When is @Factory useful?

- Run the same test class for multiple roles, products, tenants, countries, or feature configurations.
- Create instances that each have their own instance fields and setup/teardown lifecycle.
- Select the objects to test dynamically instead of hard-coding separate classes for every scenario.

2. Explain the uploaded Sample1 example

Your source defines a class called Sample1 with two test methods. The factory example creates three separate Sample1 objects.

```
public class Sample1 {

    @Test(priority = 1)
    public void login() {
        System.out.println("Login TC");
    }

    @Test(priority = 2)
    public void home() {
        System.out.println("Home TC");
    }
}
```

Line-by-line explanation

Element	What it does
public class Sample1	Declares the test class.
@Test	Marks a method as a TestNG test method.
priority = 1 / 2	Gives a relative priority to the methods. Lower values generally run first among comparable methods; do not use priority as a substitute for proper test dependencies.
System.out.println(...)	Prints a message, useful in a learning example but usually replaced by assertions and structured logging in a real test.

Factory creates three objects

```
public class TestFactory {
    @Factory
    public Object[] abc() {
        Sample1 obj1 = new Sample1();
        Sample1 obj2 = new Sample1();
        Sample1 obj3 = new Sample1();

        Object[] execute = new Object[3];
        execute[0] = obj1;
        execute[1] = obj2;
        execute[2] = obj3;

        return execute;
    }
}
```

Each call to new Sample1() creates a distinct object. Each object has its own instance fields. The factory then returns all three references in the array.

Count the tests: Sample1 contains 2 @Test methods and the factory returns 3 instances: 2 methods x 3 instances = 6 test-method invocations, assuming both methods are enabled and no configuration excludes them.

3. FactoryDemoTest: returning different test classes

Your second source example returns three Sample1 instances plus one Sample2 instance. TestNG discovers all test methods on each returned object.

```
public class Sample2 {
   @Test public void testCases3() {
      System.out.println("Test cases 3");
   }
   @Test public void testCases4() {
      System.out.println("Test cases 4");
   }
   @Test public void testCases5() {
      System.out.println("Test cases 5");
   }
}

public class FactoryDemoTest {
   @Factory
   public Object[] abc() {
      Sample1 obj1 = new Sample1();
      Sample1 obj2 = new Sample1();
      Sample1 obj3 = new Sample1();
      Sample2 obj4 = new Sample2();

      return new Object[] { obj1, obj2, obj3, obj4 };
   }
}
```

Count all available test-method invocations

Returned objects	Methods in each class	Potential invocations
3 x Sample1	2 methods per Sample1	3 x 2 = 6
1 x Sample2	3 methods in Sample2	1 x 3 = 3
Total	All methods enabled	6 + 3 = 9

What to remember

- The factory can return objects from more than one test class, as shown in your source.
- The factory method itself normally acts as an instance provider; the @Test methods live on the returned classes.
- Do not assume a global order such as all Sample1 tests finishing before any Sample2 test. The exact order depends on TestNG method ordering and configuration.
- If classes are public, place each public top-level class in its own Java source file with the matching file name.

Important distinction: Creating 4 objects does not mean only 4 tests run. TestNG runs the @Test methods found on every returned object.

4. Use constructors to pass different test data

One major use of @Factory is creating test instances with different constructor arguments. This is useful when each instance should represent a distinct scenario.

Example: test the same product workflow with different products

```
public class ProductScenarioTest {
    private final String productName;
    private final String addButtonId;

    public ProductScenarioTest(String productName, String addButtonId) {
        this.productName = productName;
        this.addButtonId = addButtonId;
    }

    @Test
    public void scenarioHasConfiguredProduct() {
        System.out.println("Product: " + productName);
        System.out.println("Button ID: " + addButtonId);
        // Real test steps use these fields to select and validate a product.
    }
}
```

Factory supplies the constructor values

```
public class ProductScenarioFactory {
    @Factory
    public Object[] createScenarios() {
        return new Object[] {
            new ProductScenarioTest("Sauce Labs Backpack",
                                    "add-to-cart-sauce-labs-backpack"),
            new ProductScenarioTest("Sauce Labs Bike Light",
                                    "add-to-cart-sauce-labs-bike-light"),
            new ProductScenarioTest("Sauce Labs Fleece Jacket",
                                    "add-to-cart-sauce-labs-fleece-jacket")
        };
    }
}
```

What happens?

1. TestNG calls the factory method.
2. The factory constructs three ProductScenarioTest objects, each with its own productName and addButtonId values.
3. TestNG runs scenarioHasConfiguredProduct() for each returned instance.
4. Each test instance reads its own final fields; the data for one object is not overwritten by the other instances.

Why final fields? The constructor assigns the scenario configuration once. Making these fields final helps prevent accidental changes after the test instance is created.

5. Practical Selenium example using @Factory

This example expands the constructor scenario into a Selenium test on the public SauceDemo sample application. It creates a separate test instance per product and a separate WebDriver session for each instance. Use this as a learning example and confirm the demo site is available before running it.

ProductScenarioTest.java

```
package tutorial10;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
public class ProductScenarioTest {
    private final String productName;
    private final String addButtonId;
    private WebDriver driver;
    private WebDriverWait wait;
    public ProductScenarioTest(String productName, String addButtonId) {
        this.productName = productName;
        this.addButtonId = addButtonId;
    }
    @BeforeClass(alwaysRun = true)
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.get("https://www.saucedemo.com/");
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
        wait.until(ExpectedConditions.visibilityOfElementLocated(
            By.className("inventory_list")));
    }
    @Test
    public void shouldAddConfiguredProductToCart() {
        driver.findElement(By.id(addButtonId)).click();
        driver.findElement(By.className("shopping_cart_link")).click();
        WebElement cartProduct = wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                By.className("inventory_item_name")));
        Assert.assertEquals(cartProduct.getText(), productName);
    }
    @AfterClass(alwaysRun = true)
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

The two constructor fields determine which product is selected. @BeforeClass creates a browser for this test instance and logs in. The @Test adds the configured product and asserts that the cart shows the expected product. @AfterClass closes the browser even when setup or a test fails.

6. Factory class and testng.xml configuration

ProductScenarioFactory.java

```
package tutorial10;

import org.testng.annotations.Factory;

public class ProductScenarioFactory {
   @Factory
   public Object[] createScenarios() {
      return new Object[] {
         new ProductScenarioTest("Sauce Labs Backpack",
                                 "add-to-cart-sauce-labs-backpack"),
         new ProductScenarioTest("Sauce Labs Bike Light",
                                 "add-to-cart-sauce-labs-bike-light"),
         new ProductScenarioTest("Sauce Labs Fleece Jacket",
                                 "add-to-cart-sauce-labs-fleece-jacket")
      };
   }
}
```

testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Factory Product Suite">
   <test name="Product Scenarios">
      <classes>
         <class name="tutorial10.ProductScenarioFactory"/>
      </classes>
   </test>
</suite>
```

Run from Maven

```
mvn test -Dsurefire.suiteXmlFiles=testng.xml
```

The XML includes the factory class, not each generated ProductScenarioTest object. When TestNG executes the factory, it receives the returned objects and discovers their test methods.

Parallel option: After the test is stable sequentially, you can try <suite name="Factory Product Suite" parallel="instances" thread-count="3">. Each returned test instance must own its own WebDriver and other mutable resources. Remove the parallel attribute to run sequentially.

7. Execution flow and object isolation

Lifecycle for the product example

5. TestNG discovers ProductScenarioFactory from testng.xml.
6. The @Factory method constructs and returns three ProductScenarioTest objects.
7. For each object, TestNG recognizes its @Test methods and lifecycle configuration methods.
8. @BeforeClass initializes that instance's ChromeDriver and logs in.
9. The test method executes using the product data stored in that object.
10. @AfterClass calls driver.quit() for that instance.

Separate objects versus separate browsers

Concept	What is isolated?	Example
Object instance	Instance fields belong to each Java object.	Each test has its own productName.
WebDriver session	A browser window/session created by a driver.	Each instance launches its own Chrome session.
Thread	An execution path used by a worker.	Parallel TestNG worker runs an instance.

A new Java object does not magically launch a browser. The browser is created only by code such as new ChromeDriver(). Similarly, @Factory itself does not guarantee parallel execution; parallelism is controlled by TestNG suite configuration.

If a test case has several dependent steps

For a flow such as login -> add to cart -> checkout -> payment, either keep the steps in one end-to-end test, or deliberately manage the authenticated state and dependencies. Avoid placing dependent steps in separate test methods and assuming factory instance ordering will always preserve a business workflow.

Practical trade-off: A separate browser per instance improves isolation, but consumes CPU, RAM, browser processes, and time. Choose thread-count based on available resources and test environment capacity.

8. @Factory vs @DataProvider vs invocationCount

Feature	@Factory	@DataProvider	invocationCount
Main purpose	Create test-class instances.	Feed multiple data rows to a test method.	Repeat one test method.
What varies?	Object/constructor configuration and instance fields.	Arguments passed to the test method.	Same method; normally same input unless test logic changes it.
Typical example	One instance per user role or product.	One login test with many username/password rows.	Repeat a test 5 times to probe stability.
Returns/uses	Factory method returns test objects, commonly Object[].	Provider method returns Object[][].	An annotation attribute on @Test.
Best when	Each scenario needs its own per-instance setup/state.	The test method is mostly the same for each data row.	You need controlled repetition, not distinct object configuration.

Choosing the right tool

- Use @Factory when constructor-level configuration or instance-level state is a natural fit.
- Use @DataProvider when the same test method needs to run for many rows, such as valid/invalid login inputs.
- Use invocationCount for repetition. Do not use it as a substitute for meaningful test data.

Example: DataProvider instead of Factory

```
@DataProvider(name = "searchTerms")
public Object[][] searchTerms() {
    return new Object[][] {
        { "backpack" },
        { "bike light" },
        { "fleece jacket" }
    };
}

@Test(dataProvider = "searchTerms")
public void searchShouldAcceptTerm(String term) {
    // This same method receives each term in turn.
}
```

With a DataProvider, the test method receives the term as a parameter. With the Factory example, each test object holds its own term/product data in fields assigned by its constructor.

9. Understanding risks in the original Selenium sample

Your uploaded Sample1 has one test method that creates the driver and logs in, followed by addToCardTest that uses the driver field and quits the browser. This is useful for learning a multi-step flow, but it couples the second method to the successful completion and execution order of the first.

Potential issues

Risk	Why it matters	Better approach
Driver setup inside loginTest	If setup fails, later tests may use a null or invalid driver.	Create and validate the driver in a lifecycle method; keep cleanup in teardown.
Tests depend on shared browser state	Running one method alone may fail because login was not run.	Make tests independent or explicitly model dependencies and setup.
Thread.sleep() calls	They wait a fixed time even if the page is ready earlier; fast/slow environments cause flakiness.	Use WebDriverWait and ExpectedConditions.
Driver close vs quit	close() closes the current window; quit() ends the full session.	Use quit() for suite/test cleanup unless only one tab should close.
Parallel runs share external state	Multiple sessions can still overwrite the same account/cart/server data.	Use unique accounts/data or isolate test records.
Too many factory objects	Each object may create another browser session and consume resources.	Keep scenario count and thread-count intentional.

If keeping the original two-method flow

At minimum, put browser initialization in a reliable lifecycle method and cleanup in an always-run teardown method. However, a multi-step checkout scenario is often clearer as one test method if later steps cannot work without earlier state.

Do not confuse: A new Sample1 object isolates Java instance fields from other Sample1 objects. It does not isolate shared static fields, shared files, shared database records, or a shared user account.

10. Best practices for @Factory frameworks

Design

- Return a clearly constructed array of test instances; name the factory method for its purpose, such as createProductScenarios().
- Pass immutable scenario values through constructors and store them in final fields.
- Keep factory creation logic small. Move large data construction into a dedicated test-data builder or configuration source.
- Use a separate test class for scenarios with genuinely different behavior rather than forcing every scenario into one generic class.
- Add test assertions. Printing output proves only that code ran; it does not prove that the application behaved correctly.

Selenium and lifecycle

- Create one WebDriver session per test instance/thread when running in parallel; avoid a shared static WebDriver.
- Always call driver.quit() in @AfterClass or another appropriate teardown method, guarded against null.
- Prefer explicit waits to Thread.sleep().
- Avoid test methods that rely on an earlier method's side effects unless the dependency is intentional and explicit.
- In parallel runs, avoid sharing mutable test objects, report objects without synchronization, or duplicate data that collides on the application.

Maintainability and CI

- Keep environment URLs and credentials in configuration or environment variables, not hard-coded production credentials.
- Start with sequential execution; enable parallel="instances" only after the suite is isolated and stable.
- Use deterministic test data and descriptive scenario names in logs so a failure can be mapped back to the constructor configuration.
- Run a small factory suite locally, then execute the same testng.xml in Maven/CI to reduce environment differences.

11. Troubleshooting checklist

Symptom	Likely cause	What to check
Factory method is not picked up	Factory class is not included in testng.xml or annotation is missing.	Check fully qualified class name, @Factory import, XML syntax.
No test methods run	Returned objects have no enabled @Test methods.	Check each returned object type and its test annotations.
Wrong constructor data	Values passed to new TestClass(...) are mismatched.	Check argument order/types and constructor assignment.
NullPointerException on driver	Driver setup did not run or failed.	Inspect @BeforeClass logs and ensure driver is assigned before use.
NoSuchSessionException	Driver session has already been quit or shared incorrectly.	Check when quit() runs and whether a static/overwritten driver is used.
Tests are flaky in parallel	Shared state/data or resource pressure.	Try sequential run, reduce thread-count, isolate test users/data.
Unexpected execution order	Assumed object-array order is also method execution order.	Review dependencies, priorities, XML parallel settings, and actual report output.
Many Chrome windows consume memory	One browser is created per object.	Reduce scenarios/thread count or choose a lighter test design.

Debugging sequence

11. Run testng.xml without parallel mode.
12. Print constructor data at the start of each test for temporary diagnosis.
13. Log instance identity and current browser URL when relevant.
14. Verify browser creation and quit are paired.
15. Enable parallel execution with a small thread-count, then scale gradually.

A useful log line: System.out.println("Scenario=" + productName + ", instance=" + System.identityHashCode(this));

12. Hands-on exercises with answers

Exercise 1: Count invocations

A class has four @Test methods. The factory returns five instances of that class. Assume every method is enabled. How many test-method invocations are available?

Answer: 4 x 5 = 20 invocations.

Exercise 2: One factory, two classes

Create two instances of SearchTest and one instance of CartTest. SearchTest has two @Test methods; CartTest has three. How many test-method invocations are available?

Answer: (2 x 2) + (1 x 3) = 7 invocations.

Exercise 3: Constructor configuration

Create a ProductValidationTest constructor that accepts productName and expectedPrice. Write a factory returning objects for Backpack, Bike Light, and Fleece Jacket. Use final fields so each object retains its scenario values.

```
public class ProductValidationTest {
    private final String productName;
    private final double expectedPrice;

    public ProductValidationTest(String productName, double expectedPrice) {
        this.productName = productName;
        this.expectedPrice = expectedPrice;
    }

    @Test
    public void productScenarioHasData() {
        Assert.assertNotNull(productName);
        Assert.assertTrue(expectedPrice >= 0.0);
    }
}
```

Factory return values: new ProductValidationTest("Backpack", 29.99), new ProductValidationTest("Bike Light", 9.99), and new ProductValidationTest("Fleece Jacket", 49.99). The numbers are illustrative sample data, not a guarantee of current application prices.

Exercise 4: Make it parallel

Register the factory class in testng.xml and add parallel="instances" with thread-count="3" only after verifying each instance creates and closes its own browser. Confirm all instances have distinct test data and do not share mutable static state.

13. Interview questions and quick revision

What is @Factory?

A TestNG annotation used on a method that returns test instances for TestNG to execute.

What should a factory method return?

Commonly Object[] containing test class objects.

Why use a factory instead of creating many test classes?

It lets one test class be configured as multiple distinct instances, often through constructor arguments.

Does @Factory automatically execute in parallel?

No. Parallel behavior is controlled through TestNG suite configuration.

How many methods execute if a class has 3 @Test methods and factory returns 4 objects?

Potentially 3 x 4 = 12 test-method invocations, assuming they are enabled and included.

What is constructor-based test data?

Passing scenario values to the test class constructor so each instance stores its own configuration.

How is @Factory different from @DataProvider?

@Factory creates test objects; @DataProvider supplies argument rows to a test method.

Why use a distinct WebDriver per test instance in parallel runs?

It prevents one test from navigating or quitting another test's session.

What does @AfterClass do in the Selenium example?

It closes the browser session for that test instance.

What is a common mistake with factories?

Assuming that returning many objects guarantees parallel execution or that object-array order dictates the complete global execution order.

Quick-reference checklist

- Factory class is listed in testng.xml.
- The @Factory method returns correctly constructed test objects.
- Constructor arguments match the intended scenario.
- Every test object owns its mutable state; each parallel instance owns its own WebDriver.
- Assertions validate behavior and teardown reliably closes resources.
- Parallel mode is configured and tested deliberately, not assumed.

Final takeaway: @Factory is most valuable when each test instance represents a distinct scenario/configuration. Keep the instance setup predictable, the test state isolated, and the assertions meaningful.